



25SC2107E – Machine Learning

Skilling Experiment 2

Titanic Survival — Preprocessing Pipeline and Cleaned Dataset

Dataset: Titanic | Course Outcome: CO1 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment 2: Titanic Survival — Preprocessing Pipeline and Cleaned Dataset

Dataset: Titanic

Course Outcome: CO1

1.1 Aim

To turn the raw Titanic table from Experiment 1 into a clean table of numbers that a machine-learning model can actually read — by removing bad columns, filling missing values, converting words into numbers and putting all numbers on the same scale — and to store every one of those steps inside a single object that can be saved and reused.

1.2 Learning Objectives

After completing this experiment, students should be able to:

- Remove columns that give away the answer or repeat another column.
- Sort columns into numeric, categorical and ordinal groups.
- Explain why the data must be split *before* any cleaning is done.
- Use `SimpleImputer`, `StandardScaler`, `OneHotEncoder` and `OrdinalEncoder`.
- Explain what a `Pipeline` and a `ColumnTransformer` do.
- Save a fitted pipeline with `joblib` so later experiments can reuse it.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib seaborn scikit-learn joblib
```

1.4 Dataset and Link

Titanic — the same 891 passengers as Experiment 1, loaded with `seaborn.load_dataset("titanic")`, which downloads <https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv>. If your lab is offline, keep a local copy of that CSV beside the notebook and use `pd.read_csv("titanic.csv")` instead.

Note

Why does raw data need cleaning at all?

A machine-learning model is just a big calculator. It can only do arithmetic. So before it sees your table:

- **Every box must contain a number.** The word "male" cannot be multiplied. It must become a number first.
- **No box may be empty.** 177 passengers have no age recorded. An empty box breaks the arithmetic.
- **The numbers should be roughly the same size.** `fare` runs from 0 to 512, while `sibsp` runs from 0 to 8. The model wrongly treats the bigger numbers as more important.

This experiment fixes all three problems. That is all “preprocessing” means.

1.5 Procedure

1. Create a notebook `skill_lab2.ipynb`; select the `.venv` interpreter.
2. Import the libraries and load the Titanic dataset.
3. Drop the columns that give away the answer or repeat another column.
4. Rebuild the `is_alone` feature and check it is correct.

5. Separate **X** from **y** and split into training and validation sets.
6. Build the three cleaning branches, one per type of column.
7. Join the branches with a **ColumnTransformer** and fit it on the training set only.
8. Put the column names back and plot **fare** before and after cleaning.
9. Write **build_titanic_pipeline()** and **validate_pipeline()**, then save the result.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```

1 import numpy as np
2 import pandas as pd
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5 import joblib
6 from sklearn.model_selection import train_test_split
7 from sklearn.pipeline import Pipeline
8 from sklearn.compose import ColumnTransformer
9 from sklearn.impute import SimpleImputer
10 from sklearn.preprocessing import StandardScaler, OneHotEncoder,
    OrdinalEncoder

```

What is happening:

- The first four imports are the same tools you used in Experiment 1.
- The new imports are of two kinds. Think of them as **machines** and **holders**.
- **Machines** do one job each to your columns:
 - **SimpleImputer** — fills empty boxes.
 - **StandardScaler** — shrinks big numbers and grows small ones so all are similar in size.
 - **OneHotEncoder** — turns words into 0/1 columns.
 - **OrdinalEncoder** — turns ranked words or numbers into 0, 1, 2 in order.
- **Holders** keep the machines organised:
 - **Pipeline** — runs machines one after another.
 - **ColumnTransformer** — sends different columns to different machines.
- **joblib** saves the finished object to a file on your computer.

Cell 2 — Load the data and drop the bad columns

```

1 titanic = sns.load_dataset("titanic")
2
3 leaks = ["alive", "who", "adult_male", "class", "embark_town",
4         "alone", "deck"]
5 df = titanic.drop(columns=leaks)
6 print(df.shape)
7 df.head()

```

What is happening: seven columns are removed. They are not all removed for the same reason.

- **alive** — **it gives away the answer.** **alive** says “yes” or “no”. **survived** says 1 or 0. They are the same fact in different words. If the model sees **alive**, it scores 100% without learning anything. This mistake is called a **data leak**.
- **class, embark_town, who, adult_male** — **they repeat a column we keep.** **class** says “First” where **pclass** says 1. Keeping both adds no new information.
- **deck** — **it is nearly empty.** 688 of its 891 boxes are blank. There is too little left to be useful.
- **alone** — **a special case.** It is none of the above. It is a column seaborn *calculated* from two other columns: it is **True** when **sibsp** + **parch** equals 0. We remove it here and rebuild it ourselves in Cell 3, so that we can practise building a feature and then *check* we built it

right.

Result: the table goes from 15 columns down to 8.

A question that catches every leak: *could I have known this before the ship sank?* The ticket price, yes. Whether the person is listed as “alive”, no.

Cell 3 — Rebuild the `is_alone` feature, then check it

```
1 df["is_alone"] = ((df["sibsp"] + df["parch"]) == 0).astype(int)
2
3 # did we rebuild seaborn's "alone" column correctly?
4 matches = (df["is_alone"] == titanic["alone"].astype(int)).all()
5 print("is_alone matches the original alone column:", matches)
6
7 numeric = ["age", "fare", "sibsp", "parch", "is_alone"]
8 categorical = ["sex", "embarked"]
9 ordinal = ["pclass"]
```

What is happening:

- `sibsp` = how many brothers, sisters or husband/wife were on board.
- `parch` = how many parents or children were on board.
- If both are 0, the passenger travelled alone. So `sibsp + parch == 0` gives True or False, and `.astype(int)` turns that into 1 or 0.
- The next two lines **check our work**. The original table is still stored in `titanic` (only `df` lost columns), so we can compare. It prints **True** — our formula is correct.
- The last three lines sort the 8 columns into three groups. **Each group will be cleaned in a different way.**

Why the check matters: normally when you build a new feature there is no answer to compare against. If your formula is wrong, nothing complains — you simply get a wrong column and later a wrong model. Here the answer already exists, so practise checking now.

Note

The three kinds of column — and why we must tell them apart

- **Numeric** — normal numbers where adding and subtracting make sense.
Example: `age`. A 40-year-old is exactly 10 years older than a 30-year-old.
- **Categorical** — names with *no* order.
Example: `embarked` (the port: C, Q or S). No port is “bigger” than another.
- **Ordinal** — names or numbers that *do* have an order, but the gaps cannot be measured.
Example: `pclass`. 1st class is better than 2nd, which is better than 3rd. But “2nd minus 1st” is not a real quantity.

How to decide, in three questions:

1. Can I subtract two values and get a sensible answer? → **numeric**
2. If not, can I put the values in a fair order? → **ordinal**
3. If not → **categorical**

Warning: the computer cannot decide this for you. `pclass` is stored as a whole number, exactly like `age`. Only you know that 1, 2, 3 are *ranks* and not *amounts*. This is why Cell 3 sorts the columns by hand.

Cell 4 — Separate X and y, then split

```
1 X = df.drop(columns="survived")
2 y = df["survived"]
3
4 X_train, X_val, y_train, y_val = train_test_split(
5     X, y, test_size=0.2, random_state=42, stratify=y)
```

```
6 print(X_train.shape, X_val.shape)
```

What is happening:

- **X** = the 8 question columns. **y** = the answer column. The answer must never hide inside the questions.
- **test_size=0.2** keeps 20% of the passengers aside. That gives **712 rows to learn from** and **179 rows to be tested on**.
- **random_state=42** makes the shuffle the same every time you run it, so your results do not change by luck.
- **stratify=y** makes sure both parts have the same mix of survivors (about 38%).

Why split *now*, before any cleaning?

- Think of the validation rows as an exam paper you have not seen.
- Suppose you fill the missing ages using the average of *all* 891 passengers. That average is partly made from exam rows.
- You have now used exam answers while studying. Your exam mark will look better than you deserve.
- So: split first, then clean using only the training rows.

Cell 5 — Branch 1 and 2: the numeric and categorical machines

```
1 numeric_pipe = Pipeline([
2     ("impute", SimpleImputer(strategy="median")),
3     ("scale", StandardScaler()),
4 ])
5
6 categorical_pipe = Pipeline([
7     ("impute", SimpleImputer(strategy="most_frequent")),
8     ("onehot", OneHotEncoder(handle_unknown="ignore")),
9 ])
```

What is happening:

- Each Pipeline is a **list of steps that run in order**, top to bottom.
- The word in quotes ("impute", "scale") is just a nickname you choose. It has no effect on the maths.

The numeric branch:

- **Step 1 — fill the gaps with the median.** The median is the middle value when the numbers are lined up in order; the mean is the average.
First, notice which columns this actually affects. Of the five numeric columns, only **age** has any gaps (177 of them). **fare**, **sibsp**, **parch** and **is_alone** are complete, so the imputer inspects them and changes nothing. In practice this step fills **age** and only **age**.
So why the median? Because the median cannot be dragged by a few unusual values, while the mean can. For **age** the two answers are in fact close (roughly 30 versus 28), so here the choice barely matters. We still choose the median because it is the *safe default*: it stays sensible for any numeric column, including a badly skewed one, whereas the mean only behaves well when the values are fairly symmetric. Pick the rule that cannot go wrong, rather than the one that happens to be fine today.
- **Step 2 — scale.** Every column is rewritten so its average becomes 0 and its spread becomes 1. Now **fare** and **sibsp** are the same size, and neither looks more important than it is.

The categorical branch:

- **Step 1 — fill the gaps with the most common value.** You cannot take the median of a word, so we use whichever port appears most often. Only 2 boxes need this.
- **Step 2 — one-hot encode.** This turns words into 0/1 columns (explained fully in Cell 7).
- **handle_unknown="ignore"** is safety. If a new passenger boarded at a port never seen before, the program writes all zeros instead of crashing.

Cell 6 — Branch 3: the ordinal machine

```

1 ordinal_pipe = Pipeline([
2     ("impute", SimpleImputer(strategy="most_frequent")),
3     ("ordinal", OrdinalEncoder(categories=[[1, 2, 3]])),
4 ])

```

What is happening:

- pclass has an order: 1st is better than 2nd is better than 3rd.
- OrdinalEncoder converts them to 0, 1, 2 while **keeping that order**.
- categories=[[1, 2, 3]] tells it the correct order directly, instead of letting it guess from whichever value it meets first.

Why not one-hot encode pclass instead?

- One-hot would make three separate yes/no columns.
- The model would then have no way of knowing that 2nd class sits *between* 1st and 3rd.
- That useful piece of information would be thrown away.

And the opposite mistake — be careful:

- Never use OrdinalEncoder on a column with no real order.
- If you coded ports as S=0, C=1, Q=2, the model would believe Q is “bigger” than C.
- The data never said that. *You* accidentally said it.

Cell 7 — Join the branches and clean the data

```

1 preprocessor = ColumnTransformer([
2     ("num", numeric_pipe, numeric),
3     ("cat", categorical_pipe, categorical),
4     ("ord", ordinal_pipe, ordinal),
5 ])
6
7 X_train_t = preprocessor.fit_transform(X_train)
8 X_val_t = preprocessor.transform(X_val)
9 print(X.shape[1], "→", X_train_t.shape[1])

```

What is happening:

- Each line inside the brackets says: (**nickname, which machine, which columns**).
- So the ColumnTransformer sends each group of columns to the correct branch, then joins all the results back together side by side.

The two most important words in this experiment:

- fit_transform(X_train) — **learn and then apply**. It works out the median age, the average fare, the list of ports ... and then cleans the training data using them.
- transform(X_val) — **apply only**. It reuses those very same numbers on the validation data. It does not learn anything new.
- **Rule to remember: fit on train, transform on both**. If you fitted on the validation data, you would be studying from the exam paper again.

Note**What is a Pipeline? What is a ColumnTransformer?**

A Pipeline is a list of steps that always run in the same order.

- Like a recipe: first chop, then fry, then serve. Never fry before chopping.
- In Cell 5 the numeric recipe is: first fill gaps, then scale.
- You give the whole recipe one command and it performs every step for you.

A ColumnTransformer sends different columns down different pipelines.

- Like sorting clothes before washing: white clothes, coloured clothes and delicate clothes each need a different wash. You would not wash them all the same way.
- Our three “washes” are the numeric branch, the categorical branch and the ordinal branch.

- After each group is cleaned its own way, the `ColumnTransformer` joins everything back into one table.

Why bother? Why not just clean the columns one by one?

- **It cannot forget a step.** Doing it by hand, it is easy to scale the training data and forget to scale the validation data.
- **It remembers the numbers it learned.** The median age, the scaling values, the list of ports are all stored inside it.
- **It can be saved as one file.** Six months later you load that one file and clean a brand-new passenger in exactly the same way. Without it you would have to remember every number by hand.

Note

How do 8 columns become 11?

Only the word columns grow. Numbers stay as they are.

Group	Column	Columns out	Why
Numeric	age	1	already a number
Numeric	fare	1	already a number
Numeric	sibsp	1	already a number
Numeric	parch	1	already a number
Numeric	is_alone	1	already 0 or 1
Categorical	sex	2	female, male
Categorical	embarked	3	C, Q, S
Ordinal	pclass	1	becomes 0, 1 or 2
Total		11	from 8

So the growth comes only from one-hot encoding. Here is what that means for `sex`:

Original sex	sex_female	sex_male
female	1	0
male	0	1
female	1	0

- One column of words becomes one column *per possible value*.
- In each row, exactly one of them is 1 and the rest are 0. “One hot” means “only one is switched on”.
- `sex` has 2 possible values, so it becomes 2 columns. `embarked` has 3 ports, so it becomes 3 columns.
- That is $2 + 3 = 5$ columns replacing 2 original ones — **3 extra columns**. And $8 + 3 = 11$.

Why do it this way? Because no port gets a bigger number than another. All the ports are treated equally — which is exactly right, since no port outranks any other.

Cell 8 — Put the names back on

```
1 names = preprocessor.get_feature_names_out()
2 clean = pd.DataFrame(X_train_t, columns=names)
3 clean.head()
```

What is happening:

- After cleaning, the result is a plain grid of numbers with **no column names**. That is hard

to read.

- `get_feature_names_out()` asks the transformer: *what did you produce?*
- Wrapping it in a DataFrame puts the names back so you can read the table again.
- Look at the names: `num_fare`, `cat_sex_female`, `ord_pclass`. The first part is the nickname of the branch it came from, so you can see exactly where each column was made.
- You should count 11 columns here — the same 11 explained in the table above.

Cell 9 — Fare before cleaning

```
1 plt.hist(X_train["fare"], bins=40)
2 plt.title("Fare - raw")
3 plt.show()
```

What is happening:

- Most passengers paid less than 50.
- A very small number paid several hundred.
- Notice the numbers along the bottom: they run from 0 to about 512. **Remember this range for the next cell.**

Cell 10 — Fare after cleaning

```
1 plt.hist(clean["num_fare"], bins=40)
2 plt.title("Fare - after imputation and scaling")
3 plt.show()
```

What is happening:

- Because of Cell 8 we can ask for the column by its name.
- **Look at the shape:** it is exactly the same as before. Same tall bar on the left, same long thin tail.
- **Look at the numbers along the bottom:** they have changed completely. Instead of 0 to 512, they now sit around 0, mostly between -1 and $+3$.

The lesson most students get wrong:

- Scaling does **not** change the shape of your data.
- Scaling does **not** remove the long tail.
- Scaling only changes the *units*, like converting kilometres to miles. The journey is the same length; only the number on the label changes.
- Its whole purpose is to stop a column being treated as important just because its numbers happen to be big.

Deliverable Function

```
1 def build_titanic_pipeline():
2     """Return the unfitted preprocessor built in Cells 5, 6 and 7."""
3     return ColumnTransformer([
4         ("num", numeric_pipe, numeric),
5         ("cat", categorical_pipe, categorical),
6         ("ord", ordinal_pipe, ordinal),
7     ])
8
9
10 def validate_pipeline(pipeline, X_train, X_val):
11     train = pipeline.transform(X_train)
12     val = pipeline.transform(X_val)
13
14     assert not np.isnan(train).any(), "missing values left in
        training data"
```



```

15     assert train.shape[1] == val.shape[1], "train and val columns
       differ"
16     assert train.shape[1] > 6, "too few features - encoders did not
       run"
17
18     print("Pipeline validation PASSED")
19     print("Features:", train.shape[1], " Train rows:", train.shape[0])
20
21
22 pipeline = build_titanic_pipeline()
23 pipeline.fit(X_train)
24 validate_pipeline(pipeline, X_train, X_val)
25
26 joblib.dump(pipeline, "titanic_preprocessor.joblib")

```

What is happening: the two functions do two different jobs.

`build_titanic_pipeline()` — **writes the recipe.**

- It collects the three branches from Cells 5, 6 and 7 into one place.
- It returns the recipe *unused*. Nothing is cleaned yet.
- This means the same recipe can be applied to any set of passengers later.

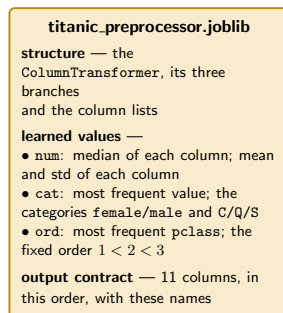
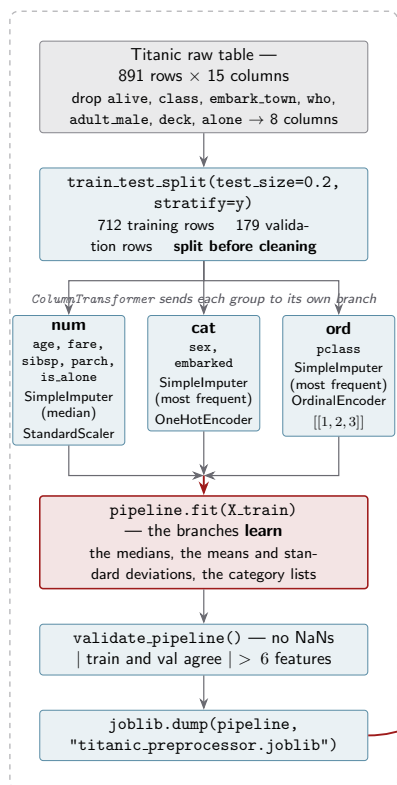
`validate_pipeline()` — **checks the result.**

- An **assert** is a small test. If the statement is false, the cell stops and prints your message. If it is true, **nothing happens at all**.
- So **silence means success**. Seeing no error is the good outcome.
- Check 1: no empty boxes are left. An empty box would break the model later.
- Check 2: training and validation have the same number of columns. If not, they disagree and the model cannot be tested.
- Check 3: more than 6 columns exist. If not, the encoders never ran.

`joblib.dump` — **saves everything to a file.**

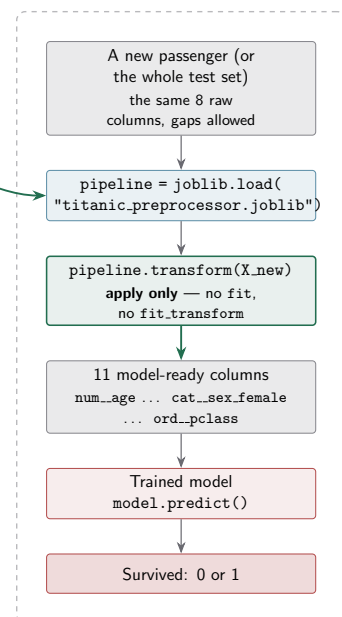
- It saves the pipeline *after* it has learned — medians, scaling values and port names all included.
- Experiment 5 will open this exact file.
- So a brand-new passenger gets cleaned using the identical numbers used during training.
- If those numbers did not match, the model would be fed data in a different shape from the data it learned on, and its answers would be wrong. This problem has a name: **training-serving skew**.

Experiment 2 — Preparation (the file is written)



Structure alone is not enough. It is the learned values that must travel with it — recomputing them on new data is training-serving skew.

Experiment 5 — Use (the file is read)



1.7 Result

The raw Titanic table was reduced from 15 columns to 8 by removing one leaking column, four repeated columns, one nearly empty column and one calculated column. The calculated column was rebuilt as `is_alone` and verified against the original. The remaining columns were sorted into numeric, categorical and ordinal groups, cleaned by a `ColumnTransformer` fitted on the training rows only, producing 11 model-ready columns. The output was checked by `validate_pipeline()` and the fitted pipeline was saved as `titanic_preprocessor.joblib`.

Note

The whole experiment in six lines

1. Throw away columns that cheat, repeat, or are empty.
2. Rebuild `is_alone`, and check it against the original.
3. Split into training and validation *before* touching anything else.
4. Sort the columns into three types, because each needs different treatment.
5. Learn the cleaning numbers from the training rows; apply them to both sets.
6. Save the fitted pipeline so every future passenger is cleaned identically.

Self-Exploration & Viva Questions

1. A classmate scales the whole dataset first and splits afterwards. Their score is higher than yours. Where did the extra marks come from, and why are they not real?
2. `age` is filled with the median, not the average. Why?
3. `pclass` uses `OrdinalEncoder` but `embarked` uses `OneHotEncoder`. What is it about the data that decides which one is correct?
4. You started with 8 columns and ended with 11. Explain where each of the 3 extra columns came from.
5. *Explore*: change `handle_unknown="ignore"` to `"error"`, then pass a passenger whose port is `"X"`. What happens? What real-life situation does this copy?